

College of Engineering
Computer Science & Eng. Dept.
Course: CMP 257-Web
Application Programming
Date: 20th March 2025



Course Instructor: Dr. Dana Dghaym
Email: ddghaym@aus.edu

Office: ESB-2067
Phone: 971-6-5154991

Homework 2: Java Programming

➤ Objectives:

- To use Java basics.
- To apply OOP concepts in Java:
 - Encapsulation-Inheritance-Polymorphism

Hand in: upload your HW on iLearn:

- One clear and organized solution document (paste all code and **screenshots of the program working**).
- Your code should be well documented with comments where appropriate.
- A zip file including all your code
- You are **NOT allowed** to use ChatGPT or any AI generative tool to generate the code and you are not allowed to copy and paste the solution from the internet, this is a violation of AUS academic integrity. However, you are allowed to seek help online to find out how things are done, but the code should be original as a result of your own working, you can include references where applicable.

Due Date: Thursday 27th March -11:59 PM. Late submissions will incur a **10% penalty per day**. The submission **link will be removed 48 hours after the deadline**. Email submissions will not be accepted.

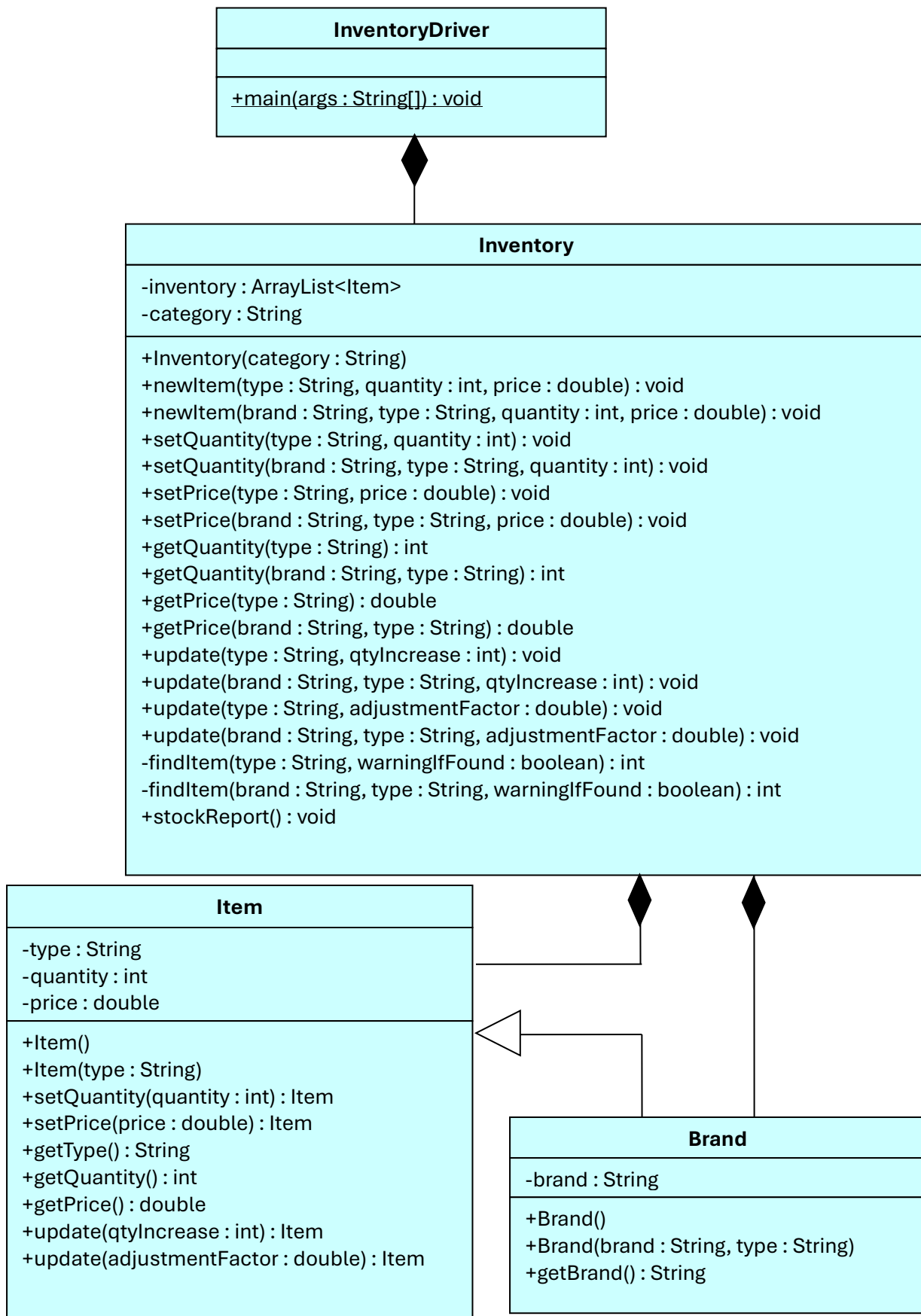
Grocery Store Inventory

As your first job fresh out of college, you are hired as a cashier at the local supermarket. In order to impress the store manager, you decide to write an Inventory program to help with inventory tracking.

Use an `Inventory` class with an `ArrayList` instance variable that will hold a list of objects, which may be either of two kinds: (1) Generic items – instances of an `Item` class that includes three instance variables for: type description, quantity of that type, and price of one unit of that type. (2) Branded items – instances of a `Brand` class that is derived from the `Item` class and adds an additional instance variable to specify a brand name.

Here is a suggested UML class diagram:

Note: The black diamond in UML represents the composition relationship which is a specific case of **association** relationship, where one class (whole) owns the parts (components).



The `findItem` method returns the index of the item if it found only one, otherwise returns -1. It also prints some appropriate messages based on the result, see the main method and the sample output for guidance.

Most of the Inventory methods should start with a call to `findItem` method. In the two `newItem` methods, start with a call to the `findItem` method with the second argument `true`. If this returns -1, add a new Item or new Brand to the inventory.

In the `setQuantity`, `setPrice`, `getQuantity`, and `getPrice` methods, start with a call to the `findItem` method with the second argument `false`. If this does not return -1, set or get the quantity or the price in or from the object at that location. Start the update methods with a call to the `findItem`, as in the set- methods above, and call the appropriate update method in the `Item` class only if `findItem` does not return -1.

Use the `InventoryDriver`'s `main` method to test your other classes, do not change the Driver class, your output should be as shown below:

```
public class InventoryDriver {

    public static void main(String[] args) {
        Inventory store = new Inventory("groceries");
        store.newItem("bread", 15, 9.99);
        store.newItem("AlMarai", "milk", 2, 2.00);
        store.newItem("eggs", 3, 1.50);
        store.newItem("bread", 2, 1.25);    // warning: in stock
        store.stockReport();
        store.update("AlMarai", "milk", .25); // raise price 25%
        store.update("eggs", -1);           // lower quantity by 1
        store.update("juice", 3);           // warning: not stocked
        store.newItem("AlRawabi", "milk", 4, 1.95);
        store.stockReport();
        store.setPrice("lizards", 99);    // warning: not stocked
        store.setQuantity("AlMarai", "milk", 3);
        store.setPrice("eggs", 2.00);
        System.out.println("milk quantity: " + store.getQuantity("AlRawabi", "milk")); // not
stocked

        System.out.println("milk quantity: " + store.getQuantity("milk"));    // ambiguity
        System.out.println("eggs price: " + store.getPrice("eggs"));
        System.out.println("milk price: " + store.getPrice("milk")); // ambiguity
    }
}
```

The output:

bread already exists

bread - in stock: 15, price: \$9.99

AlMarai milk - in stock: 2, price: \$2.00

eggs - in stock: 3, price: \$1.50

Total value: \$158.35

Cannot find juice

bread - in stock: 15, price: \$9.99

AlMarai milk - in stock: 2, price: \$2.50

eggs - in stock: 2, price: \$1.50

AlRawabi milk - in stock: 4, price: \$1.95

Total value: \$165.65

Cannot find lizards

milk quantity: 4

Found more than one brand of milk

milk quantity: -1

eggs price: 2.0

Found more than one brand of milk

milk price: -1.0